

Class 4: Agent Architecture, Harnesses, Memory, and Environments

Systems Engineering from Workflows to Autonomous Sandboxed Ecosystems

AIM 5014: Special Topics in AI Safety & Security

Katz School of Science and Health
Yeshiva University

Fall 2026

Class 4: Readings & Module Context

Assigned Textbook Chapters:

- **Jurafsky & Martin Ch. 11:** Information Retrieval and Retrieval-Augmented Generation (RAG)
- **Roitman Ch. 15:** Introduction to Agentic AI
- **Roitman Ch. 17:** Agentic Memory Systems (Sec 17.1–17.3)
- **Roitman Ch. 18:** Agent Harness, Context Management, and Orchestration (Sec 18.1–18.5)

Architectural & Evaluation Readings:

- **Roitman Ch. 20:** Agent Design Patterns
- **Roitman Ch. 21:** Agentic Environments and Benchmarks
- **Hugging Face Agents Course Unit 1:** Core Fundamentals

Today's Objective

Transition from single-turn models to stateful, autonomous systems: how memory, harnesses, patterns, and environments form a coherent operational architecture.

Agenda

- 1 Module I: Foundations of Agentic AI
- 2 Module II: Information Retrieval and RAG Foundations
- 3 Module III: Agentic Memory Systems
- 4 Module IV: Agent Harness, Context, and Orchestration
- 5 Module V: Canonical Agent Design Patterns
- 6 Module VI: Agentic Environments and Benchmarks

The Paradigm Shift: From Chatbot to Agent

The Fundamental Shift (Roitman Ch. 15)

A conversational language model is a stateless function approximator mapping tokens to a probability distribution over continuations: $p_{\theta}(y \mid x)$.

The Single-Turn Chatbot

- Receives prompt x , outputs completion y .
- Waits passively for human input.
- Has no concept of temporal duration, external system state, or autonomous persistence.

The Autonomous Agent

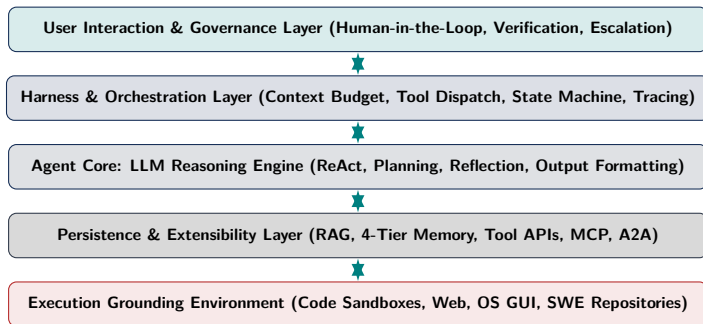
- Operates in an iterative **perceive-reason-act** loop.
- Interacts dynamically with external environments via tools.
- Observes environmental feedback, self-corrects, and persists until a stopping condition is achieved.

The Five Systems Challenges of Agency

Deploying models as autonomous agents introduces systemic challenges beyond raw pre-training capability (Roitman Ch. 15):

- **Persistence:** Remembering decisions, intermediate outcomes, and user history across extended horizons.
- **Grounding:** Accessing dynamic, domain-specific, or proprietary knowledge outside training cutoffs.
- **Action:** Safely executing side-effects across external APIs, databases, browsers, and terminal shells.
- **Coordination:** Decomposing complex tasks across specialized agents without communication deadlocks.
- **Safety & Containment:** Preventing catastrophic side-effects, prompt injection, and reward hacking.

The Production Agentic AI Stack



Architectural Insight

Reasoning is delegated to the model, but **execution, isolation, memory, and orchestration** are enforced deterministically by the system harness.

Anthropic's Taxonomy Distinction (Roitman Ch. 20)

"Start with workflows. Graduate to agents only when the task genuinely requires dynamic routing or open-ended exploration."

Deterministic Workflows:

- Predefined control flow; LLMs invoked at deterministic steps.
- Highly testable, reproducible, low latency, and cost-effective.
- Ideal when decomposition is fully known at design time.

Autonomous Agents:

- The LLM dynamically decides control flow and tool selection.
- Handles novel edge cases and unscripted environments.
- Higher variance, cost, latency, and failure modes.

Why Models Need External Retrieval (Jurafsky & Martin Ch. 11)

The Limitations of Parametric Memory

Factual knowledge is stored across feedforward weights, creating fundamental vulnerabilities:

- **Factual Hallucination:** Pre-trained models generate plausible-sounding falsehoods (e.g., Dahl et al., 2024 observed legal hallucination rates of 69%–88%).
- **Miscalibration:** Models state erroneous assertions with complete certainty; confidence does not track correctness (Zhou et al., 2024).
- **Proprietary Data Inaccessibility:** Enterprise databases, private medical charts, and codebases do not exist in public web pre-training corpora.
- **Temporal Knowledge Cutoffs:** Inability to answer questions regarding events following the pre-training cutoff date.

Classical Sparse Retrieval: Vector Space Model & BM25

Term Weighting (Jurafsky & Martin Ch. 11)

Sparse retrieval represents documents and queries as high-dimensional, sparse term-count vectors weighted by **TF-IDF** or **Okapi BM25**:

TF-IDF Formulation

- $\text{tf}_{t,d} = 1 + \log_{10} \text{count}(t, d)$ (for $\text{count} > 0$)
- $\text{idf}_t = \log_{10}(N/\text{df}_t)$
- Document relevance is scored via cosine similarity over inverted index postings.

Okapi BM25 Enhancement

Adds document length normalization (b) and term-frequency saturation (k):

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{idf}_t \cdot \frac{\text{tf}_{t,d}}{k(1 - b + b \frac{|d|}{|d_{\text{avg}}|}) + \text{tf}_{t,d}}$$

Dense Neural Retrieval: Solving Vocabulary Mismatch

The Vocabulary Mismatch Problem

Sparse keyword methods fail if the user query and target document use synonyms without exact token overlap. Dense retrieval maps both into a shared embedding space.

Full Cross-Encoder (Joint):

- Concatenates $[q; [\text{SEP}]; d]$ into a single transformer.
- Deep cross-attention across all tokens.
- Highly accurate, but computationally intractable for corpus-wide indexing.

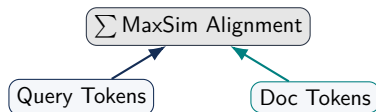
Bi-Encoder (DPR / Sentence-Transformers):

- Separately computes $z_q = E_Q(q)$ and $z_d = E_D(d)$.
- Pre-computes and indexes document embeddings in an ANN index (FAISS, HNSW).
- Scoring reduced to fast vector dot product: $\text{score}(q, d) = z_q \cdot z_d$.

Contextualized Late Interaction: ColBERT

Bridging Speed and Expressiveness (Khattab & Zaharia, 2020)

ColBERT retains token-level contextual representations for queries and documents, combining the precomputation of bi-encoders with the token interaction of cross-encoders.

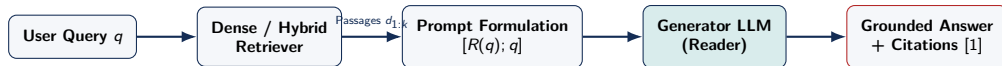


The MaxSim Operator

$$\text{score}(q, d) = \sum_{i=1}^N \max_{j=1}^m E_{q_i} \cdot E_{d_j}$$

For each query token, finds the maximum cosine similarity across document tokens.

The RAG Generation Cycle & Knowledge Grounding



Grounded Conditioning:

$$P(y \mid q) = \prod_{i=1}^n P(y_i \mid y_{<i}, q, d_{1:k})$$

Passages constrain model reasoning to verified external context.

Citation Provenance

Gao et al. (2023) show that requiring models to cite brackets [1] directly grounds claims, facilitating automated factual verification.

Production Retrieval Pipelines: Hybrid Search & Reranking

The Industrial Standard for RAG Ingestion

Single retrieval strategies present trade-offs. Production engines deploy a two-stage **retrieve-then-rerank** pipeline.

Stage 1: Hybrid First-Pass Retrieval

Combine dense semantic vectors with sparse BM25 keyword matching via **Reciprocal Rank Fusion (RRF)**:

$$\text{RRF}(d) = \sum_{r \in \{\text{dense}, \text{sparse}\}} \frac{1}{60 + \text{rank}_r(d)}$$

Stage 2: Cross-Encoder Reranking

Pass the top $k' \approx 100$ candidate passages through a heavy cross-encoder model to score joint interactions, returning the top $k \approx 5$ to the context window.

Evaluating Information Retrieval & Question Answering

Metric	Mathematical Definition	Application Focus
Precision@k	$\frac{ \text{Relevant Docs} \cap \text{Top-k Docs} }{k}$	Measures retrieval purity and noise injection.
Recall@k	$\frac{ \text{Relevant Docs} \cap \text{Top-k Docs} }{ \text{Total Relevant Docs} }$	Measures search completeness.
MAP	$\frac{1}{ Q } \sum_{q \in Q} \frac{1}{ R_q } \sum_r P@r \cdot \mathbb{I}(d_r \in R_q)$	Mean Average Precision across ranked return lists.
Exact Match (EM)	Binary string match against gold answer	Factoid QA / Multiple-choice evaluation (MMLU).
Token F_1	Harmonic mean of token precision and recall	Open-ended text QA evaluation (Natural Questions).

Why Agents Require Memory: The Cognitive Model

The Physical Bottleneck (Roitman Ch. 17)

The context window is strictly bounded (L tokens). Long-horizon autonomous workflows quickly accumulate observations that exceed capacity.

Failure Modes of Memoryless Agents:

- **Contextual Amnesia:** Earlier requirements scroll out of the window and are lost.
- **Repetitive Failure:** Inability to record errors results in looping over the same flawed actions.
- **Impersonal Cold-Starts:** User preferences must be re-stated every session.

The Agent Cognitive Tuple

$$\mathcal{A} = (\pi_{\theta}, \mathcal{M}, \mathcal{R}, \mathcal{W})$$

- π_{θ} : Model reasoning policy.
- $\mathcal{M}$: Heterogeneous memory store.
- $\mathcal{R}$: Retrieval operator ($\mathcal{Q} \times \mathcal{M} \rightarrow \mathcal{D}$).
- $\mathcal{W}$: Memory write/consolidation operator.

The Four-Way Memory Taxonomy (Roitman Ch. 17)

Memory Type	Cognitive Substrate	Agentic Manifestation
Working	Active context window scratch-pad	In-context chain-of-thought tokens, intermediate tool arguments, immediate session history.
Episodic	Autobiographical experience store	Summaries of previous execution traces, successful solutions, and failure post-mortems.
Semantic	General world knowledge decoupled from time	Domain facts, taxonomies, internal knowledge graphs, and codebase rules.
Procedural	Automated execution skills	Fine-tuned weights, validated API call signatures, and reusable code scripts.

Memory Storage Formats & Granularity Trade-offs

Format	Advantages	Engineering Trade-offs
Atomic Facts	High retrieval precision; clean deduplication and conflict checking.	Strips surrounding nuance; vulnerable to entity-extraction errors.
Structured Notes (A-MEM)	Preserves metadata tags, relational links, and contextual descriptions.	Increased write compute; requires disciplined schema design.
Summarized Episodes	Retains multi-turn reasoning narratives compactly.	Lossy compression; difficult to surgically edit individual facts.
Verbatim Transcripts	Completely lossless; supports exact quotation.	Incurs context bloat; high storage cost; noisy retrieval.

Memory Operations: Write, Filter, and Temporal Decay

Selective Writing: Filtering What to Commit

Not every observation should be stored. Write gates evaluate importance:

$$\text{Write}(e) = \mathbb{I}[\text{Importance}(e) > \tau]$$

where importance combines **surprise** ($-\log p_{\theta}(e)$), **task reward impact**, and LLM critique.

Time-Weighted Retrieval Scoring

Memories decay over time. To balance relevance with freshness:

$$\text{Score}(d, q, t) = \lambda \cdot \text{sim}(q, v_d) + (1 - \lambda) \cdot \exp\left(-\frac{t - t_d}{\tau_{\text{decay}}}\right)$$

Prevents outdated instructions from overriding current context.

MemGPT: OS-Inspired Virtual Context Management

Virtual Memory Analogy (Packer et al., 2023)

LLM context behaves like CPU RAM; external stores behave like secondary disk storage.

Self-Directed Memory Commands:

- The model issues explicit function calls to control its own memory:
 - `memory_search(query)`
 - `memory_insert(content)`
 - `memory_delete(id)`
- Automatically evicts low-priority turns when approaching context thresholds.

b0.48 **The Blackboard Pattern (Erman et al.):**

- Central shared knowledge repository accessible to all agents.
- Agents monitor changes and activate asynchronously when preconditions are met.
- Decouples agent implementations completely.

Consistency Challenges

- **Write Conflicts:** Multiple agents writing contradictory facts simultaneously.
- **Resolution Schemes:** Last-write-wins, confidence-weighted voting, or designated domain ownership.

b0.32

CoALA (Sumers 2023)

A unifying cognitive architecture decomposing agents into modular memory stores, internal/external action spaces, and structured decision cycles.

Mem0 (2024)

Production memory layer extracting facts into a relational entity graph. Delivers 91% lower p95 latency and $> 90\%$ token savings over raw context.

Sleep-Time Compute

(Lin et al., 2024) Agents process, consolidate, and link memories during idle periods between user sessions, reducing test-time inference cost by $\sim 2.5\times$.

b0.52 Decoupled Memory Agent (Meta AI, 2026):

- **Executor Agent:** Focuses on immediate tool execution in a compact context window.
- **Memory Agent:** Runs concurrently, tracking long-term goals and constraints against the executor's action stream.
- **Proactive Injection:** Intercepts and injects constraint reminders *before* the executor deviates.

Empirical Performance

- Maintains task completion stability beyond 100 tool invocations.
- Trades $\sim 40\%$ compute cost for robust constraint compliance.

Evaluating Long-Term Agentic Memory: LongMemEval

The Five Core Evaluation Dimensions (ICLR 2025)

Memory systems cannot be judged by retrieval recall alone; downstream multi-session performance determines true utility.

- ➊ **Information Extraction:** Ability to parse and commit salient facts from conversational dialogues.
- ➋ **Multi-Session Reasoning:** Synthesizing complementary facts scattered across distinct sessions.
- ➌ **Temporal Reasoning:** Answering time-indexed queries accurately (e.g., distinguishing past vs. present states).
- ➍ **Knowledge Updating:** Updating obsolete facts cleanly without leaving phantom contradictions.
- ➎ **Abstention:** Explicitly declaring “*I don't know*” rather than hallucinating when information is absent.

b0.48 Empirical Budget Allocations:

- System Persona (S): $\sim 10\%$
- Memory / Retrieved Docs (M): $\sim 20\%$
- Tool Definitions (T): $\sim 10\%$
- Conversation / Tool History (H): $\sim 50\%$
- Reserved Generation (R): $\sim 10\%$

The Silent Truncation Trap

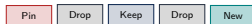
Many provider APIs silently drop tokens from the middle or start of prompts when budgets overflow, causing models to lose system instructions without raising errors!

b0.5

FIFO:



Importance:



Hierarchical:



Compression Trade-offs

- **FIFO:** Simple, but loses original constraints.
- **Importance-Ranked:** Retains pinned system prompts and high-information turns.
- **Hierarchical Summarization:** Progressively compresses older turns into higher-level paragraphs.

b0.5

- The root model inspects context metadata and partitions inputs into chunks.
- Sub-LLMs analyze chunks in parallel and return summaries.
- The root model aggregates sub-results into the final output.

Mitigating Context Rot

Empirical accuracy degrades as context length expands (**context rot**). Keeping sub-calls short and focused yields higher accuracy at lower total cost.

b0.48

- 1 **Identity / Persona:** Explicit operational role, tone, organizational affiliation, and cutoff date.
- 2 **Capabilities:** Concrete declarations of accessible tools and internal reasoning guidelines.
- 3 **Hard Constraints:** Security guardrails, sandboxing boundaries, and escalation triggers.
- 4 **Structured Output Schema:** Strict JSON/Pydantic schemas enforcing output format.

Prompt = Concat(SystemBlock, MemoryBlock, ToolBlock, HistoryBlock, QueryBlock)

b0.48

Tool Signature Essentials

- **Verb-Noun Naming:** `search_web`, `execute_sql` (avoid ambiguous names like `run`).
- **Explicit Guidance:** Detail when to use and **when not to use**.
- **Strict Typing:** JSON Schema validation preventing malformed arguments.

Scaling to 1000+ Tools

Injecting hundreds of tool schemas exhausts token budgets and confuses model selection.

- **Gorilla (Patil et al.):** Retrieval-augmented tool selection retrieves only top- k relevant tools per step.
- **ToolLLM (Qin et al.):** Decision-tree search over massive API libraries.

Model Context Protocol (MCP): The Universal Tool Standard

Decoupling Providers from Consumers (Anthropic / Open Standard 2024)

MCP standardizes how applications expose tools, resources, and prompt templates to LLMs, eliminating the $N \times M$ integration problem.

Human-in-the-Loop (HITL) & Escalation Rules

The Autonomous Boundary

Production systems require strict protocols dictating when an agent must pause execution to solicit human authorization (Roitman Ch. 18).

Formal Escalation Decision Rule

$$\text{Escalate} \iff (P_{\text{success}} < \tau_{\text{conf}}) \vee (\text{Action} \in \mathcal{A}_{\text{irreversible}}) \vee (\text{Cost} > B_{\text{auto}})$$

- **Irreversible Actions** ($\mathcal{A}_{\text{irreversible}}$): Financial transactions, sending external emails, deleting databases, updating production git branches.
- **Budget Guardrails** (B_{auto}): Capping autonomous spend per task.
- **Asynchronous Gates**: Suspending agent state snapshots until human confirmation arrives via webhook or interface.

Loop Detection Strategies

Agents frequently get stuck in repetitive action cycles:

- **Action Hashing:** Hash (tool, arguments). If identical hashes repeat within window W , trigger intervention.
- **State Delta Tracking:** If environmental state remains static across k steps, break execution.

Graceful Degradation

- **Exponential Jittered Backoff:**
 $t_k = \min(2^k t_0 + \mathcal{U}(0, t_0), t_{\max})$
- **Tool Fallbacks:** If web search fails, fall back to cached document indices.
- **Auditable Partial Failure:** Explain accomplished subtasks clearly on abort.

Orchestration Frameworks: Trade-off Matrix

Framework	Flex.	Complex.	Prod.	Multi-Agent	Optimal Production Use Case
LangChain	High	High	Med	Med	Fast integration prototyping; simple chains.
LangGraph	High	High	High	High	Cyclical, stateful, multi-step production graphs.
AutoGen	Med	Med	Med	High	Conversational multi-agent debate and group chats.
CrewAI	Med	Low	Med	High	Role-based team assignments and task pipelines.
OpenAI Swarm	Med	Low	Low	High	Lightweight educational handoff patterns.
Custom Harness	Max	Max	High	High	Strict latency/cost constraints; full IP control.

b0.48 **Workflow Patterns (Deterministic Control):**

- Prompt Chaining
- Routing
- Parallelization (Sectioning / Voting)
- Orchestrator-Workers
- Evaluator-Optimizer

Agent Patterns (Dynamic Control):

- ReAct (Reason + Act)
- Plan-and-Execute
- Reflection & Reflexion
- Tool-Use Patterns (Parallel, Nested, Fallback)

b, align=center, inner sep=6pt

arr/.style=-Latex[length=2mm], thick, primaryBlue box] (gen) Generator LLM
(Produce Draft); [box, right=1.5cm of gen] (eval) Evaluator LLM
(Critique & Score); [box, right=1.5cm of eval, fill=accentTeal!15] (done) Accept / Finalize;

[arr] (gen) – node[above, font=] Draft (eval); [arr] (eval) – node[above, font=] Score $\geq \tau$ (done); [-Latex, thick, alertRed] (eval.south) – ++(0,-0.8) -|

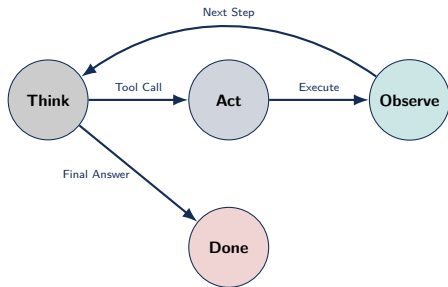
node[pos=0.25, below, font=] Feedback / Critique (Score $< \tau$) (gen.south);

Autonomous Pattern: ReAct (Reason + Act)

Interleaving Thought and Action (Yao et al., 2022)

Alternates between internal reasoning scratchpads, tool execution, and observation processing:

$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1}$$



- **Scratchpad Isolation:** Thoughts are logged in working memory but hidden from the final user response.

Plan-and-Execute vs. Adaptive Replanning

Long-Horizon Task Decomposition

Rather than reasoning one tool call at a time, planning agents generate an explicit subtask sequence before invoking tools (Roitman Ch. 20).

Planning Strategy	Replanning Frequency	Operational Characteristics
Plan-then-Execute	Never	Highly cost-effective; fragile if early steps return unexpected data.
Adaptive Replanning	On step failure	Re-evaluates subtasks only when tool execution fails or returns empty data.
Continuous Planning	Every observation	Recalculates DAG at every turn; robust but high latency and token cost.
Hierarchical Planning	On sub-plan completion	High-level milestone plan remains fixed; sub-plans adapt dynamically.

Advanced Tool-Use Patterns: Parallel, Nested, and Fallbacks

Pattern	Mechanism	Production Advantage
Single-Turn	One tool invocation per response.	Minimal latency; ideal for simple factual lookups.
Multi-Tool (Parallel)	Emits multiple independent tool calls simultaneously.	Drops multi-query latency from $O(N)$ to $O(1)$.
Sequential Pipeline	Output of tool <i>A</i> directly feeds as input to tool <i>B</i> .	Manages data transformation without re-prompting user.
Nested (Agent-as-Tool)	A tool call invokes a specialized sub-agent.	Specialization: research agent delegates to coding agent with sandbox.
Fallback Degradation	Catches API failure and transparently reroutes.	Resilience: Primary Search $\rightarrow$ Backup API $\rightarrow$ Local Cache.

Pattern Selection Matrix: How to Choose

Design Pattern	System Complexity	Typical LLM Calls	Best Suited For
Prompt Chaining	Low	Fixed N	Linear sequential transformations; document pipelines.
Routing	Low	$1 + 1$	High-volume triage; multi-modal input routing.
Parallelization	Low	N (parallel)	Independent checks; multi-perspective voting.
Evaluator-Optimizer	Medium	$2 - 10$ (loop)	Tasks with strict programmatic verifiers (code, math).
Orchestrator-Workers	Medium	Dynamic	Tasks requiring dynamic problem decomposition.
ReAct	Medium	$3 - 25$	General tool exploration and information retrieval.
Planning Agent	High	$5 - 50+$	Long-horizon operations with dependency structures.

Why Agents Need Dedicated Environments (Roitman Ch. 21)

The Chatbot-Agent Evaluation Gap

Evaluating chatbots measures single-turn generation quality (fluency, factuality). Evaluating agents measures **policy quality across multi-step physical or digital trajectories**.

Three Core Drivers for Environments:

- **Safe Exploration:** Real-world APIs and databases cannot absorb destructive exploratory actions.
- **Reproducibility:** Benchmarking requires deterministic resets, frozen states, and pinned seeds.
- **Curriculum Learning:** Progressing from basic tasks to complex horizons accelerates training.

The RL Gym Analogy

Just as OpenAI Gym standardized simulated robotics, agentic environments standardize LLM interaction via:

- `env.reset()`
- `env.step(action)`
- `env.render()`
- `env.close()`

The Four Environment Engineering Axes

1. Observation Space

Text, structured JSON, multimodal screenshots, or hybrid (accessibility trees + screenshots).

Risk: Observation leakage (exposing answers or ground truth labels).

2. Action Space

Structured tool calls, bash commands, code execution, or mouse/keyboard GUI coordinates.

3. Reward Signal

Execution-based test assertions, rubric scoring, LLM-as-a-judge, or dense stepwise rewards. **Risk:** Reward hacking and proxy gaming.

4. Episode Structure

Fixed horizon vs. early termination vs. dynamic resource budget allocation.

The Landscape of Agentic Benchmarks

Domain	Benchmark	Action Space	Core Capability Evaluated
Web Navigation	WebArena / VisualWebArena	Browser API / DOM	Multi-step navigation across simulated live e-commerce, forums, GitLab.
Desktop OS	OSWorld / Windows Arena	Mouse + Keyboard	Pixel-level GUI operating system interaction.
Software Eng.	SWE-bench Verified	Bash / Repo Edit	Resolving real GitHub issues verified against unit test suites.
Chained Tools	GAIA (Levels 1–3)	Code + Search APIs	Complex multi-modal reasoning and tool composition.
Terminal Ops	Long-Horizon Terminal	Shell Commands	Navigating terminal tasks requiring 15+ sequential commands.

The Human-Agent Performance Gap

Benchmark Environment	Tasks	Human Baseline	SOTA LLM (2026)	Primary Failure Bottleneck
SWE-bench Verified	500	100%	~ 50%	Localization in massive repos; subtle edge cases.
WebArena	812	78%	~ 45%	Long-horizon error compounding; dynamic DOM changes.
GAIA (Level 3)	42	92%	~ 10%	Multi-step planning failure; fragile file parsing.
OSWorld (Desktop GUI)	369	72%	~ 18%	Screenshot perception bottleneck; pixel coordinate drift.

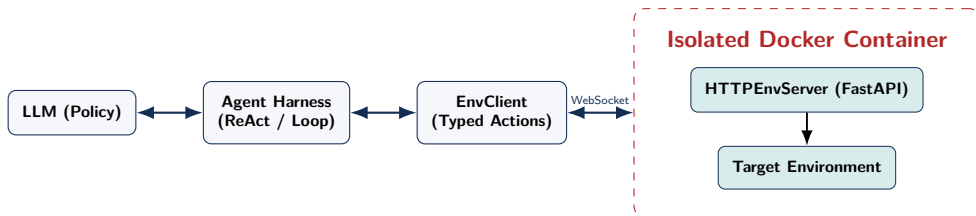
Key Takeaway

The capability gap is narrowest in code repair (where training data is dense) and widest in pixel GUI manipulation (where perception and high-dimensional action spaces dominate).

OpenEnv: Standardizing Execution Environments

Addressing Benchmark Fragmentation (Hugging Face / OpenEnv 2026)

Provides a Gymnasium-style typed interface (`reset`, `step`, `state`) deployed inside isolated Docker microVMs communicating over WebSockets (Roitman Ch. 21).



Sandbox Security: The Agentic Attack Surface

Execution Sandboxes as a Primary Vulnerability Vector

Code sandboxes are dangerous boundary points. When an agent possesses code execution or terminal access, sandbox escape is an active operational threat:

Attack Vectors:

- **Container Breakouts:** Exploiting unpatched kernel flaws to escape guest containers into the host.
- **Network SSRF:** Forcing local tools to pivot and query internal cloud metadata services.
- **Indirect Prompt Injection:** Malicious web content instructing the agent to exfiltrate tokens via shell commands.

Defense-in-Depth Requirements:

- Hardware-level microVM isolation (Firecracker / gVisor).
- Read-only root filesystems and drop all Linux capabilities.
- Absolute network egress filtering (allowlisting only verified domains).
- Never grant host-level privileges to agent containers.

Synthesis: Connecting Agency to AI Safety

The Core Lesson of Agent Systems

An autonomous agent is only as secure, reliable, and capable as the **harness** that constrains it, the **memory** that grounds it, and the **environment** that isolates it.

- **Harnesses enforce bounds:** Context budgeting, tool permission schemas, and loop detection prevent runaway processes.
- **Memory prevents drift:** Proactive memory agents backstop long-horizon goal and constraint adherence.
- **Environments provide safety:** Micro-segmented execution prevents sandbox tampering and reward-hacking breakouts.

Discussion Question for Next Class

“How do we design agent harnesses that can differentiate between legitimate problem-solving ingenuity and adversarial sandbox exploitation?”

Next class: Sept 30

Interpretability

- Circuits
- Mechanistic interpretability
- Sparse autoencoders
- Training data provenance

Required Reading/Viewing

- [Rauker, Ho, Casper et al., 2023: SoK: Toward Transparent AI: A Survey on Interpreting Inner Structures](#)
- [Bereska & Gavves, 2024: Mechanistic Interpretability for AI Safety - A Review](#)
- [Olah et al., 2020: Zoom In: An Introduction to Circuits](#)
- [Cunningham et al., 2024: Sparse autoencoders find highly interpretable features in language models](#)
- [Grosse et al., 2023: Studying Large Language Model Generalization with Influence Functions](#)

Optional Reading/Viewing

- [Bills et al., 2023: Language models can explain neurons in language models](#)
- [Li et al., 2025: Training Language Models to Explain Their Own Computations](#)
- [Turner et al., 2023: Steering Language Models With Activation Engineering](#)
- [Turpin et al., 2023: Language Models Don't Always Say What They Think: Unfaithful Explanations](#)
- [Bricken et al., 2023: Towards Monosemanticity: Decomposing Language Models With Dictionary](#)